ARTIFICIAL INTELLIGENCE lecture 3

Prof.Dr: Yusria Abu El-naga

Dr: Dina Awny

Dr: Ayman Soliman

Chapter 3 problem solving



AGENDA

- SEARCH ALGORITHMS
- UNINFORMED SEARCH

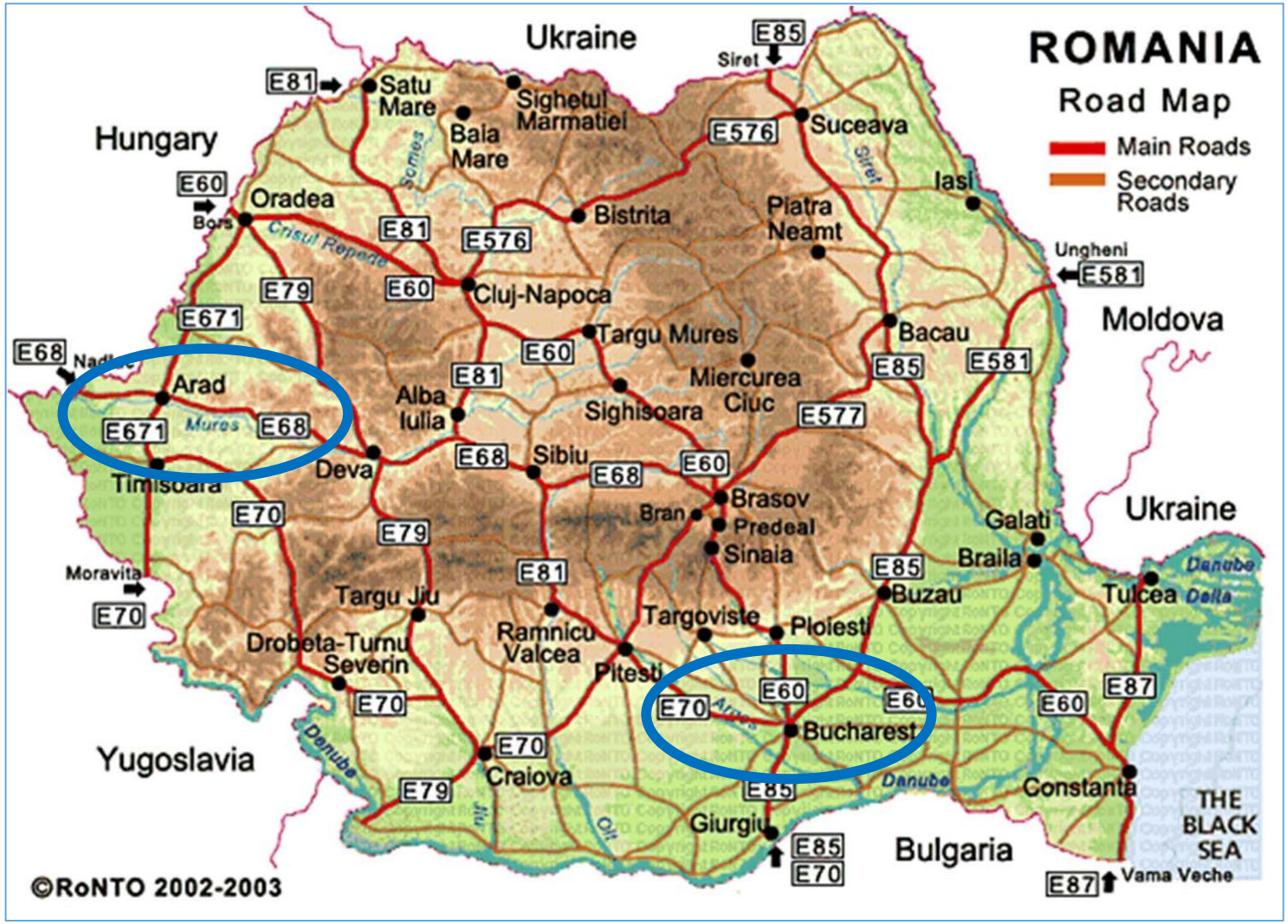
Goal

- The theory and technology of building agents that can plan ahead to solve problems
- These problems characterized by having many states.
- Example: navigation problem where there are many states and you need to pick the right choice now and in all the following states streaming together a sequence of actions that guarantee to reach the goal.



Not covered

- Navigation in fog is a different problem where the environment is partially observable, and the possible paths are unknown .



Problem

- Find A Route From Arad To Bucharest

Problem solving by searching

Requirements of searching:

- **Define Problem**
  - Represent the search space by states.
  - Define the actions the agent can perform.
  - Define the costs associated with the defined actions
- **Define a goal:** What is the agent searching for?
- **Define the solution:**
  - The goal itself?
  - The path (i.e. sequence of actions) to get to the goal?

Problem solving by searching cont

Assumptions:

- Goal-based agent.
- Environment:
  - Fully observable.
  - Deterministic.
  - Sequential.
  - Static.
  - Discrete.
  - Single agent.

Problem definition (formulation)

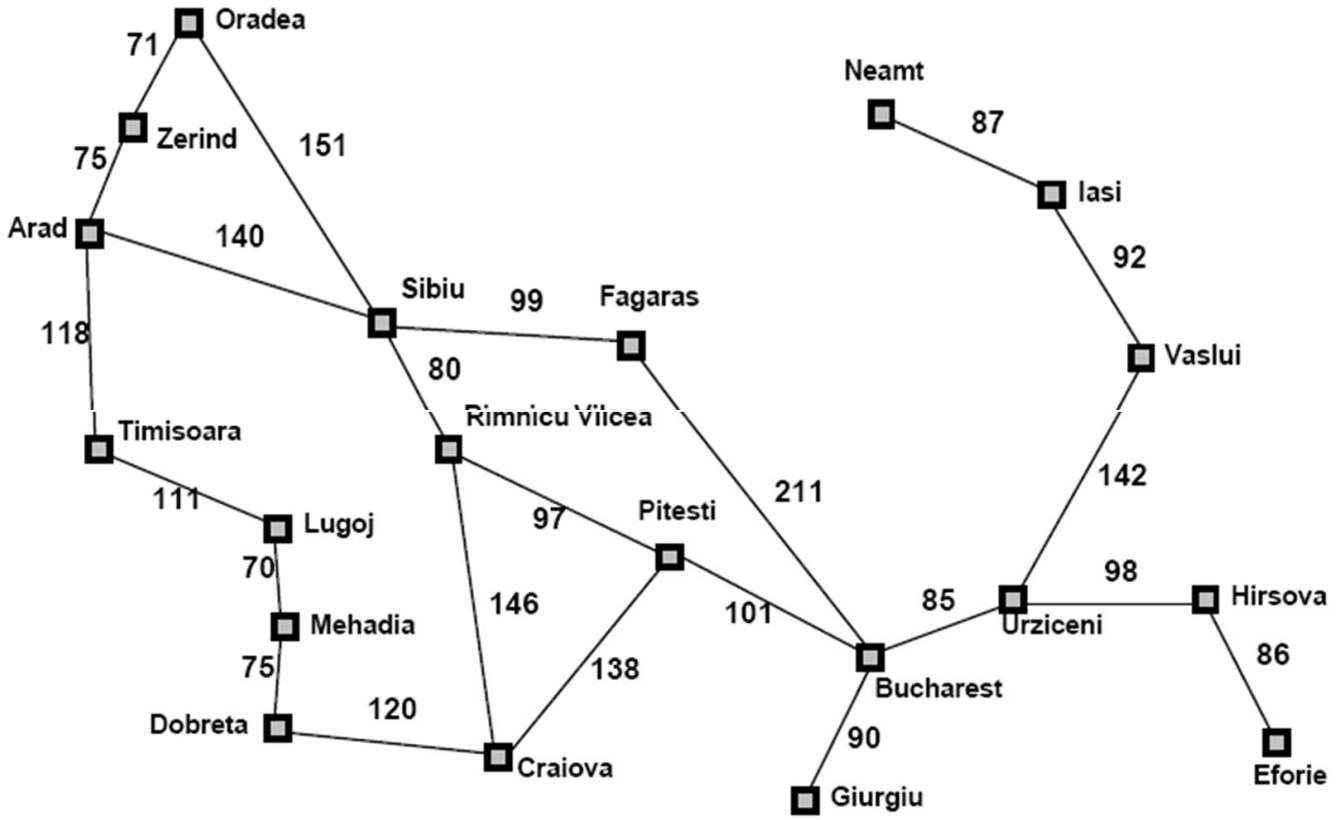
- A problem can be defined formally by five components:

1) The initial state that the agent starts in.
   INITIAL-STATE : STATE
2) The possible actions available at each state.
   ACTIONS : STATE ➔ ACTION SET
3) The transition model describing what each action does.
   RESULT : STATE x ACTION ➔ STATE
4) The goal test that detects the goal state.
   GOAL-TEST : STATE ➔ BOOLEAN
5) The path cost expressed in terms of step cost.
   STEP-COST : STATE x ACTION ➔ REAL+.

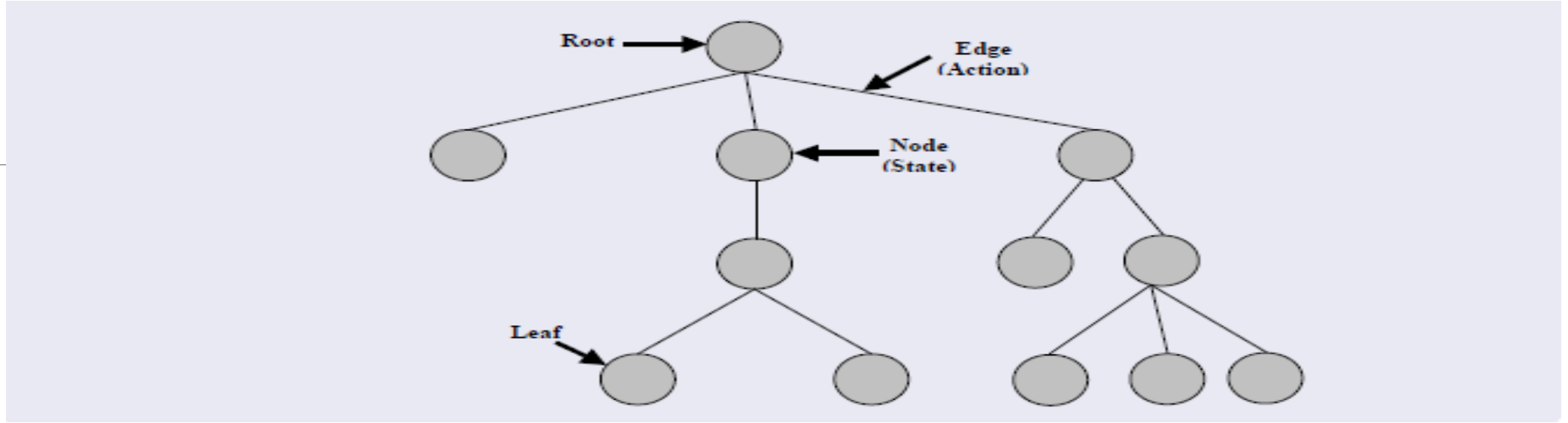
- This is known as a STATE-SPACE problem formulation

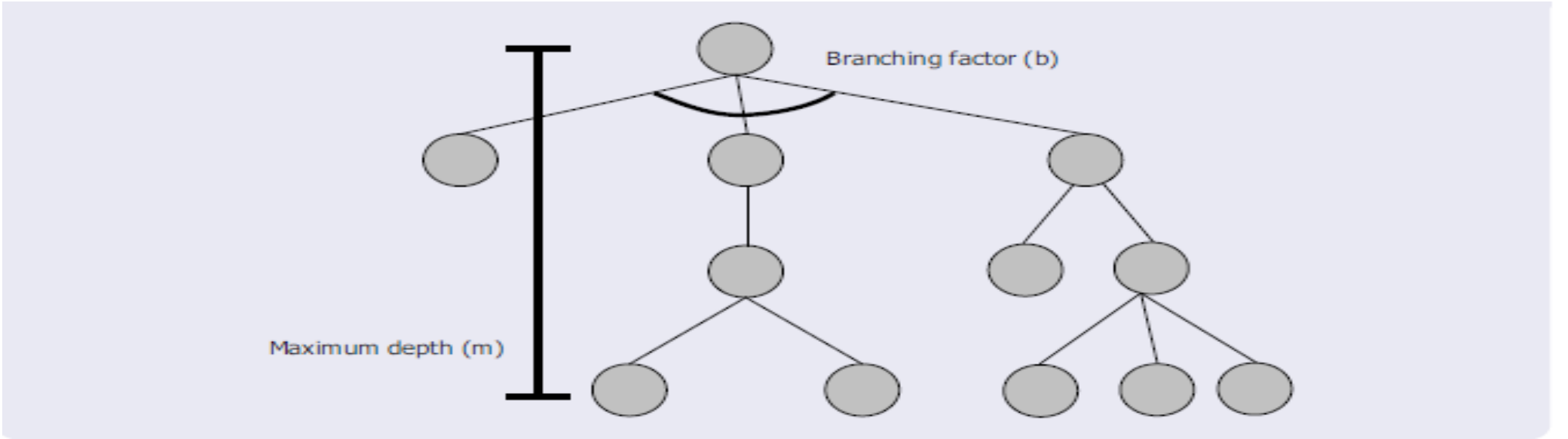
Problem definition

- After every action we separate states out into 3 parts:
  - End of the paths (Frontier states)
  - Explored states
  - Unexplored states



A search tree





General tree-search algorithm

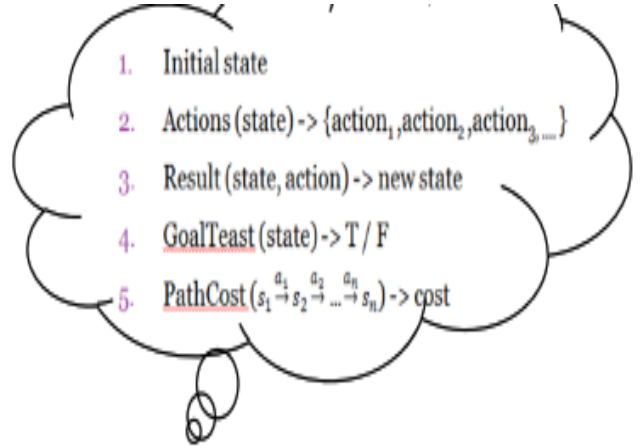
TREE-SEARCH(problem) **returns a solution, or failure**

initialize the frontier using the initial state of a problem

loop do

- **if the frontier is empty then return failure**
- choose a leaf node and remove it from the frontier
- **if the node contains a goal state then return the solution**
- expand the chosen node, adding the resulting node to the frontier

end




```
1. TreeSearch(problem)
2.
3.      frontiers={initial}
4.
5.      loop:
6.              if frontier is empty: return Fail
7.
8.              path = frontiers.remove()       //choose frontier
9.              s=path.end
10.
11.             if s is a goal: return path     // test goal
12.
13.             for a in actions(s):            //expand the path
14.                     path.end（result(s,a) )
15.                     frontiers.add(path)
```

Search Algorithms

- The method in which a search algorithm traverses the tree is known as the **search strategy**.
- The search strategy is defined by the method used by the algorithm for choosing the next node from the frontier

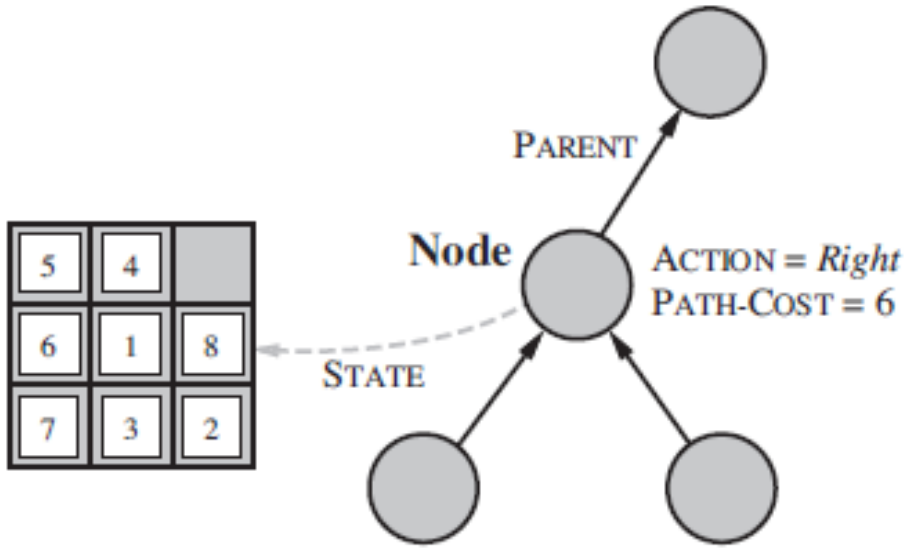
Properties of search algorithms

- **Completeness**: Is the algorithm guaranteed to find a goal node, if one exists?
- **Optimality:** Is the algorithm guaranteed to find the best goal node, i.e., the one with the cheapest path cost?
- **Time complexity:** How many nodes are generated?
- **Space complexity:** What is the maximum number of nodes stored in memory?

Search Algorithms

Infrastructure for search algorithms

- Special data structures are needed to represent: the **problem**, the **nodes**, the **frontier**, and the **explored** states.
- Each node n has the following components:
  - n.STATE, n.PARENT, n.ACTION, n.PATH-COST,
  - n.CHILD-NODE(problem, action), and n.SOLUTION()



Search Algorithms

Infrastructure for search algorithms

- Each problem p has the following components:
  - p.INITIAL-STATE, p.ACTIONS(state), p.RESULT(state,action), p:GOAL-TEST(state), p:STEP-COST(state,action).
- Frontier f is represented as a (FIFO, LIFO, or priority) queue that has the following components:
  - f:EMPTY?(), f:INSERT(element), and f:POP()
- Explored states are kept track of using a data structure that acts as a set.

Search Algorithms

Types of search algorithms

- **Uninformed Search:**
  - Only has the information provided by the problem
  - formulation (initial state, available actions, transition model, goal test, and step/path cost),
- **Informed Search:**
  - Has additional information that allows it to judge the promise of an action, i.e. the estimated cost from a state to a goal.

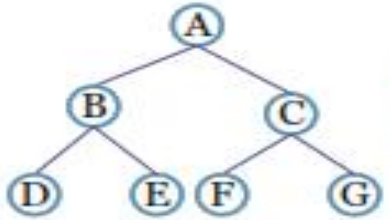
Uninformed Search

Uninformed Search algorithms

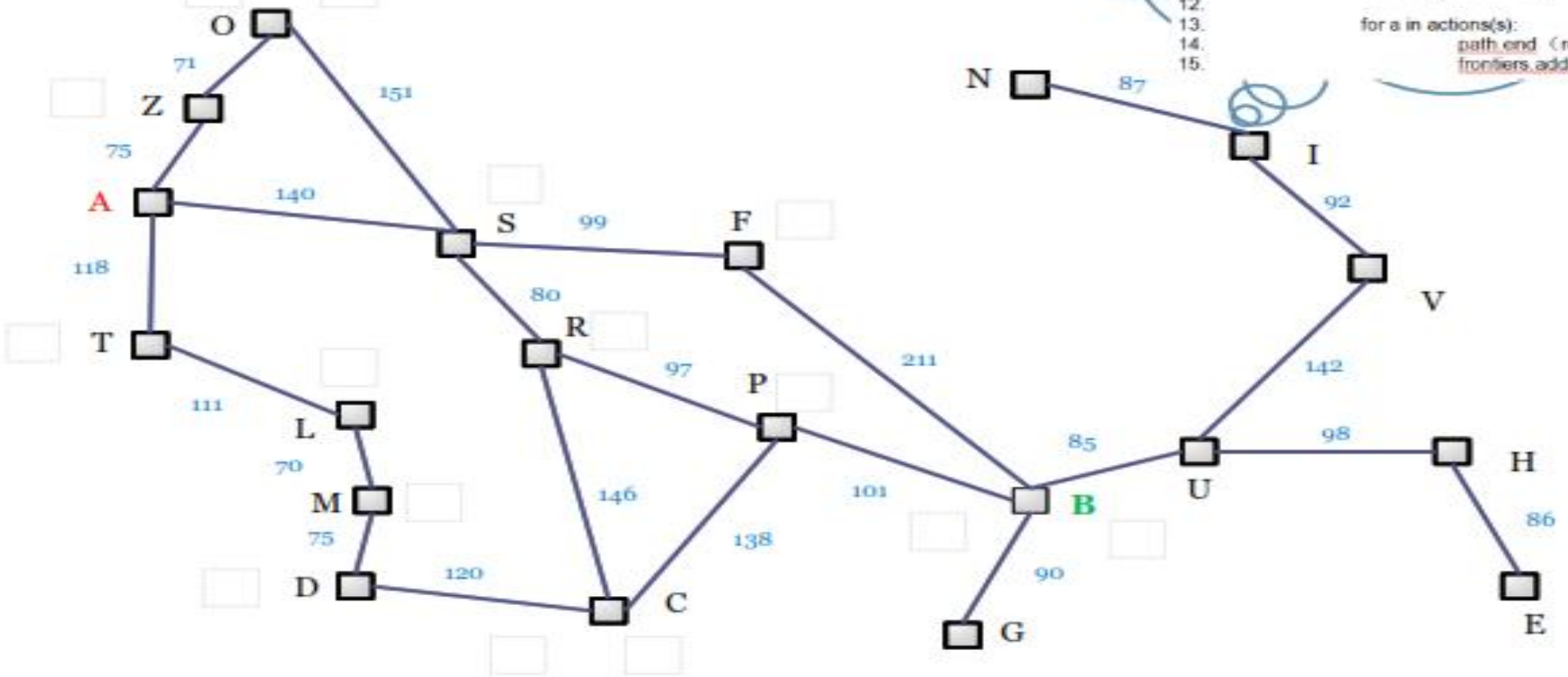
- Breadth-first search (BFS),
- Uniform-cost search,
- Depth-first search (DFS),
- Depth-limited search,
- Iterative deepening search (IDS),

Breadth First Search

- Shortest path first search



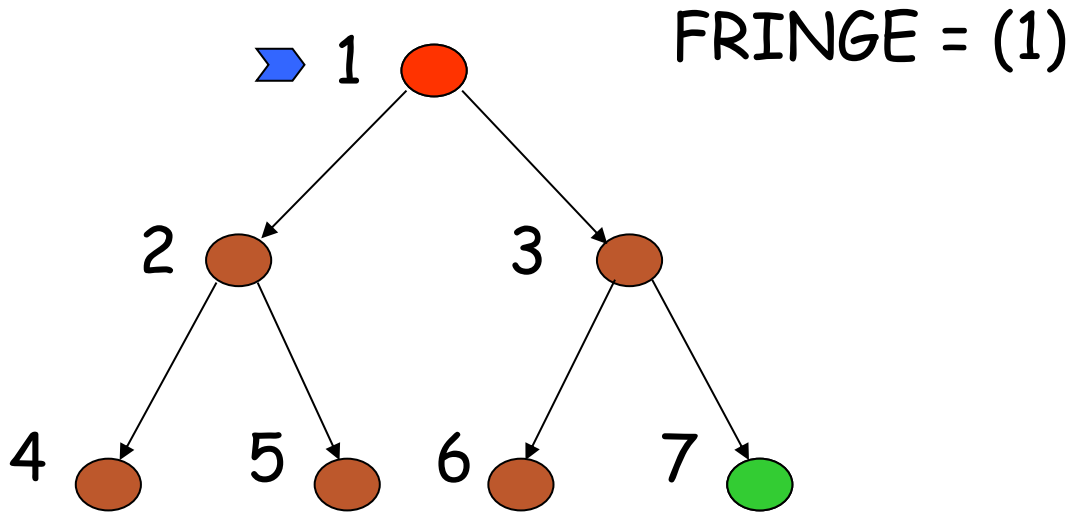

```
1. TreeSearch(problem)
2.
3.        frontiers={initial}
4.
5.        loop:
6.                if frontier is empty: return Fail
7.
8.                path = frontiers.remove()          //choose frontier
9.                s=path.end
10.
11.               if s is a goal: return path        // test goal
12.
13.               for a in actions(s):               //expand the path
14.                       path.end (result(s,a) )
15.                       frontiers.add(path)
```



1. Breadth-first search (BFS)

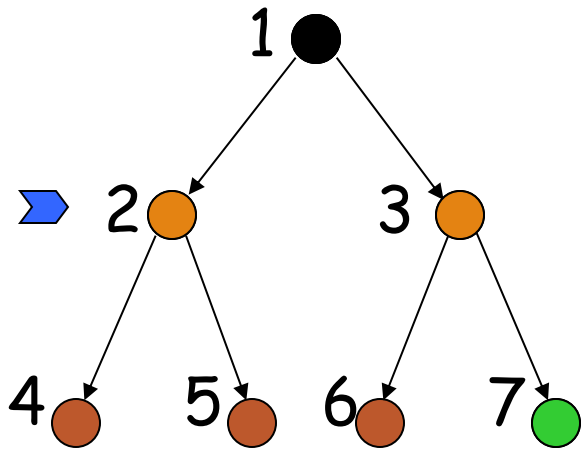
- The frontier is implemented as a FIFO queue,
- The tree is traversed on a level-by-level basis.

New nodes are inserted at the end of queue (FRINGE)



Breadth-First Strategy

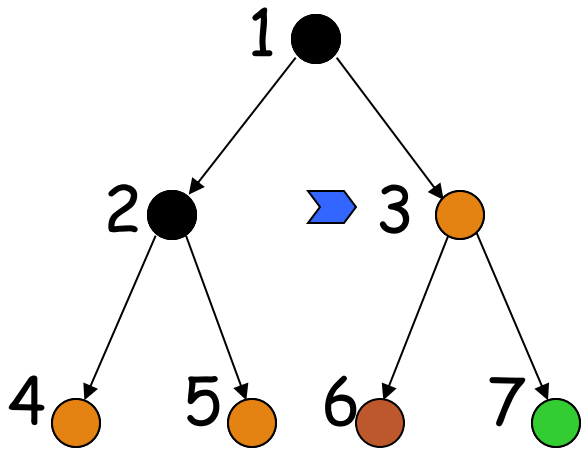
New nodes are inserted at the end of FRINGE



FRINGE = (2, 3)

Breadth-First Strategy

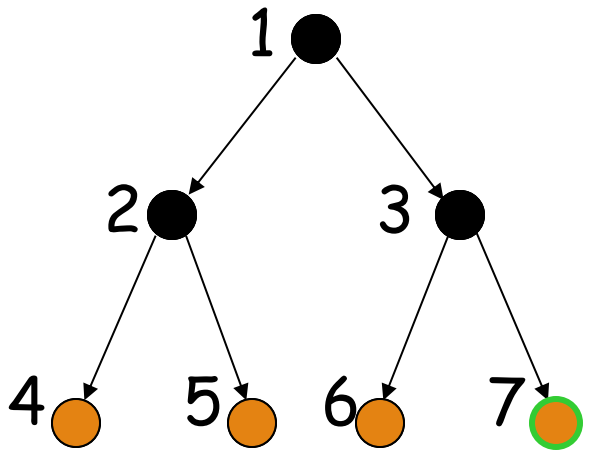
New nodes are inserted at the end of FRINGE



FRINGE = (3, 4, 5)

Breadth-First Strategy

New nodes are inserted at the end of FRINGE



FRINGE = (4, 5, 6, 7)

Breadth-first search (tree version)

function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure

```
node ← a node with STATE=problem.INITIAL-STATE, PATH-COST=0
if problem:GOAL-TEST(node.STATE) then return node.SOLUTION()
frontier ← a FIFO queue with node as the only element
loop do
    if frontier.EMPTY?() then return failure
    node ← frontier.POP() /* choose shallowest node in frontier */
    for each action in problem:ACTIONS(node.STATE) do
        child ← node:CHILD-NODE(problem, action)
        if problem:GOAL-TEST(child.STATE) then return child.SOLUTION()
                frontier:INSERT(child)
```

Breadth-first search (Graph version)

To avoid repeated paths we replace the tree search by a graph search

function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure

---

```
node ← a node with STATE=problem.INITIAL-STATE, PATH-COST=0
if problem:GOAL-TEST(node.STATE) then return node.SOLUTION()
frontier ← a FIFO queue with node as the only element
explored ← an empty set
loop do
    if frontier.EMPTY?() then return failure
    node ← frontier.POP() /* choose shallowest node in frontier */
    add node.STATE to explored
    for each action in problem:ACTIONS(node.STATE) do
        child ← node:CHILD-NODE(problem, action)
        if child.STATE is not in explored and not in frontier then
            if problem:GOAL-TEST(child.STATE) then return child.SOLUTION()
                frontier:INSERT(child)
```

Uninformed Search

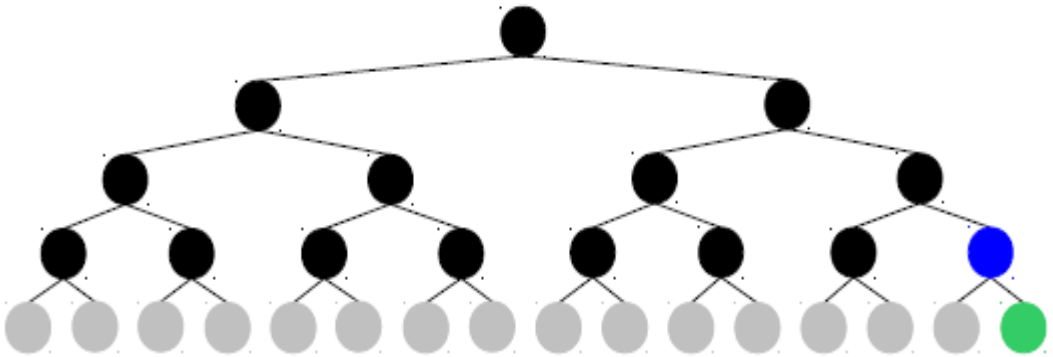
BFS properties:

Upper-bound case: when the goal node is the last node at depth d:

- Goal is detected once goal node is generated.
- Number of nodes generated:

  $b + b^2 + b^3 + \ldots\ldots + b^d = O(b^d)$.

- Space and time complexity: all generated nodes.



Uninformed Search

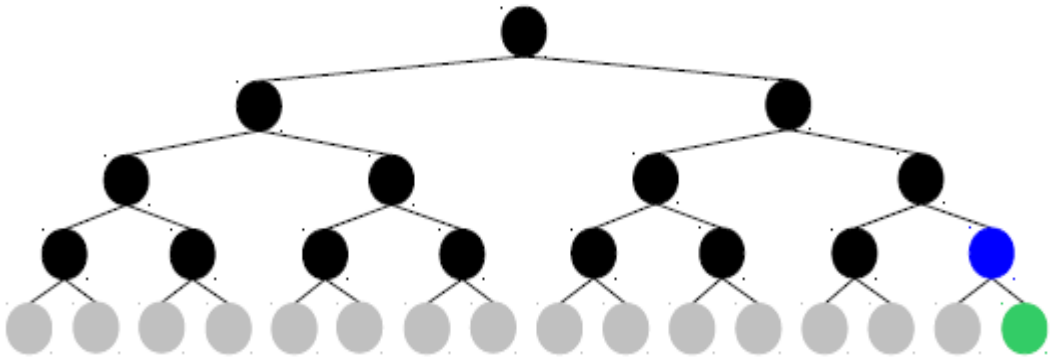
BFS properties:

Upper-bound case: when the goal node is the last node at depth d:

- Goal is detected once goal node is generated.
- Number of nodes generated:

  $b + b2 + b3 + \ldots\ldots + bd = O(b^d)$.

- Space and time complexity: all generated nodes.



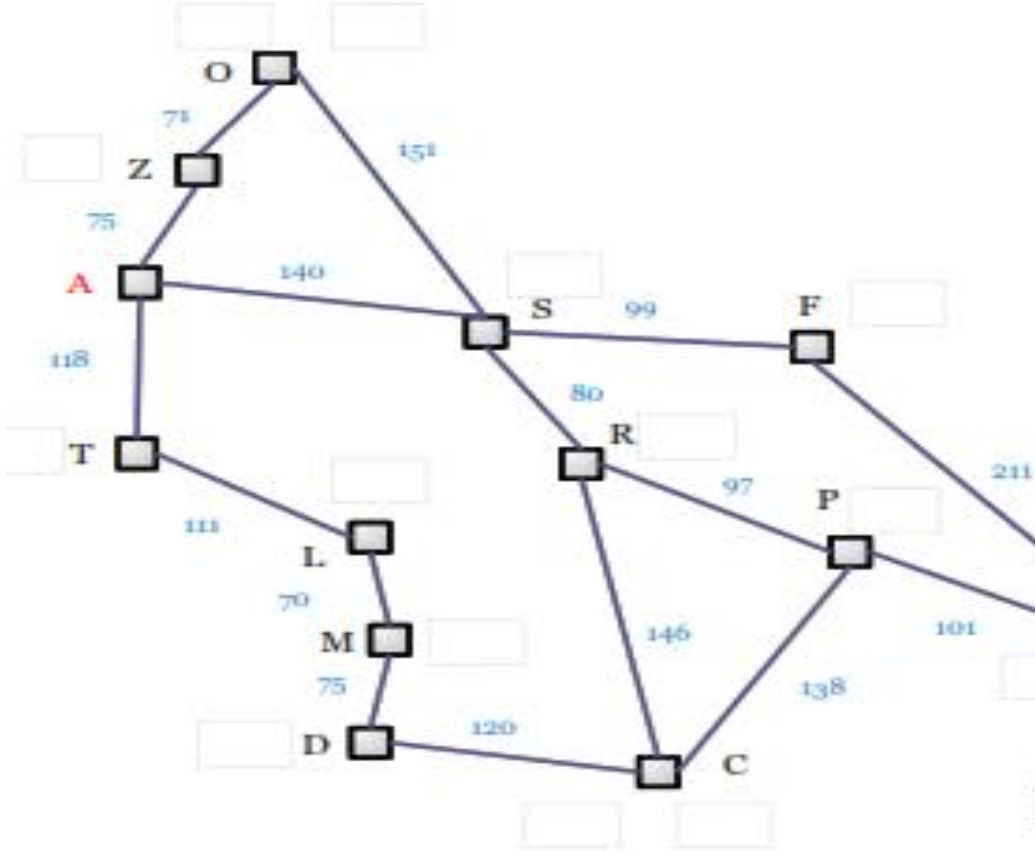
BFS properties:

- Complete (if b is finite).
- Optimal, if path cost is equal to depth:
- Guaranteed to return the shallowest goal (depth d).
- Time complexity = $O(b^d)$.
- Space complexity = $O(b^d)$

Uniform-cost search

- A simple extension of BFS that works for any step-cost function.
- Instead of expanding the shallowest node, UCS expands the node n with the lowest path cost g(n).
- The frontier is implemented as a priority queue ordered by g.
- Condition: No zero-cost or negative-cost edges.
- UCS algorithm differs from BFS algorithm in 2 aspects:
  - Goal test is applied to a node when it is selected for expansion.
  - In graph version, a test is added in case a better path is found to a node currently on the frontier.

Cheapest first search



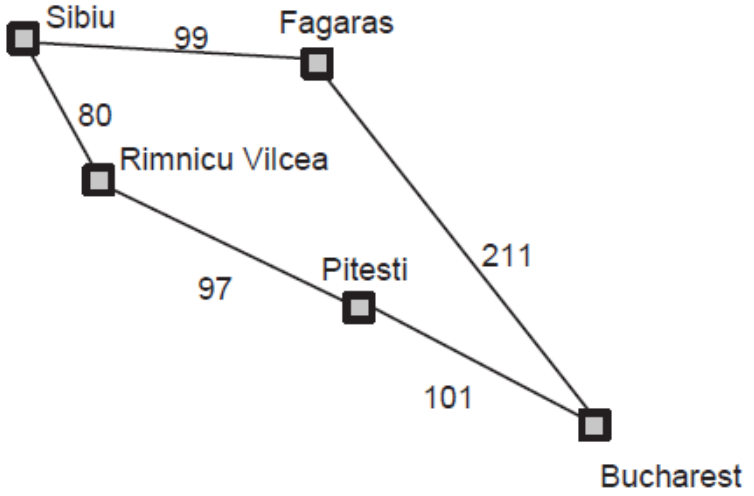

```
1. GraphSearch(problem)
2.
3.      frontiers=[initial], explored={}
4.
5.      loop:
6.          if frontier is empty: return Fail
7.
8.          path = frontiers.remove()          //choose frontier
9.          s=path.end; explored.add(s)
10.
11.         if s is a goal: return path        // test goal
12.
13.         for a in actions(s):               //expand the path
14.             if result(s,a) ∈ explored and path.end(result(s,a))
15.                 path.end(result(s,a))
16.                 frontiers.add(path)
```

Uniform-cost Search

Let g(n) be path cost of node n.

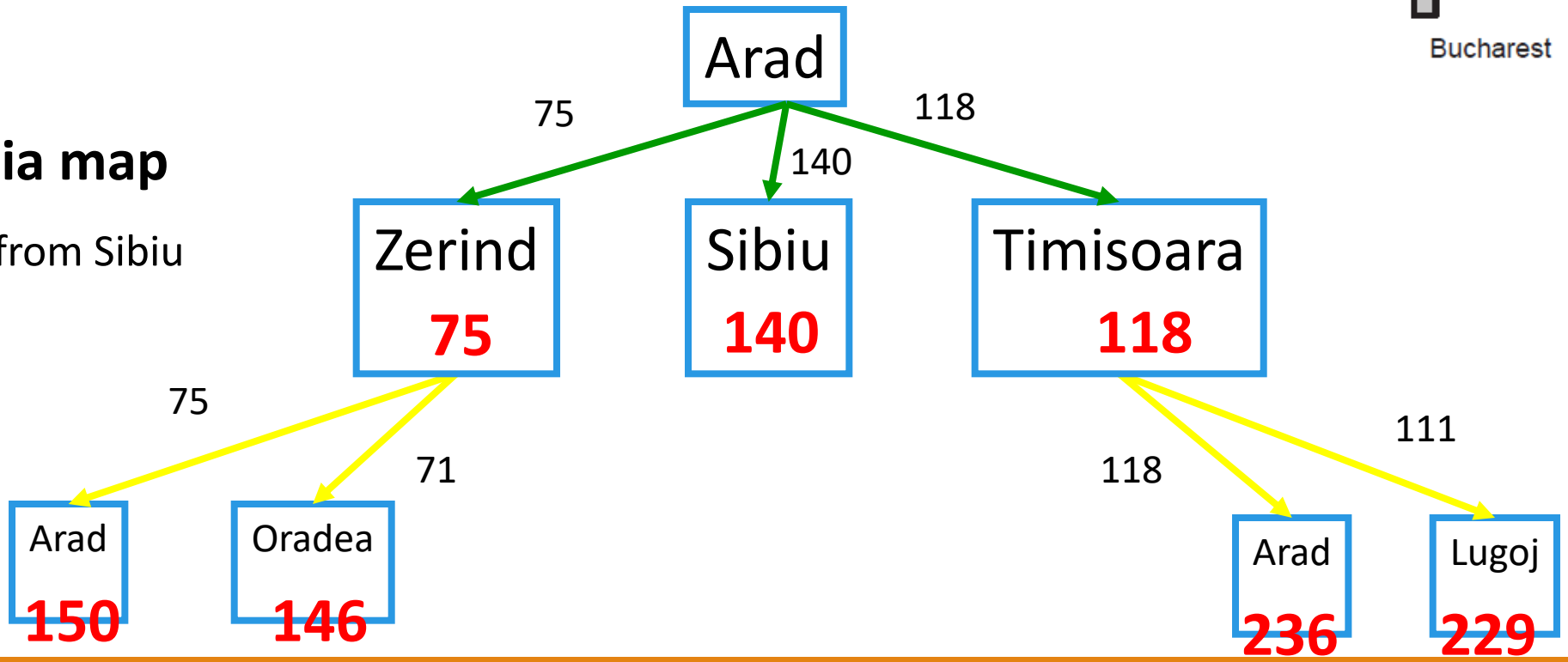
Expand least-cost unexpanded node

Implementation:
QueueingFn = Keep Fringe sorted in increasing path length order



UCS example - Romania map

Find the shortest path route from Sibiu to Bucharest.



Complete: Yes if arc costs bounded below by $\varepsilon > 0$.

Optimal: Yes

Time: # of nodes with g(n) $\leq$ cost of optimal solution

Space: # of nodes with g(n) $\leq$ cost of optimal solution

Note: Breadth first is equivalent to Uniform-Cost Search with edge cost equal a constant.

Uniform-cost search

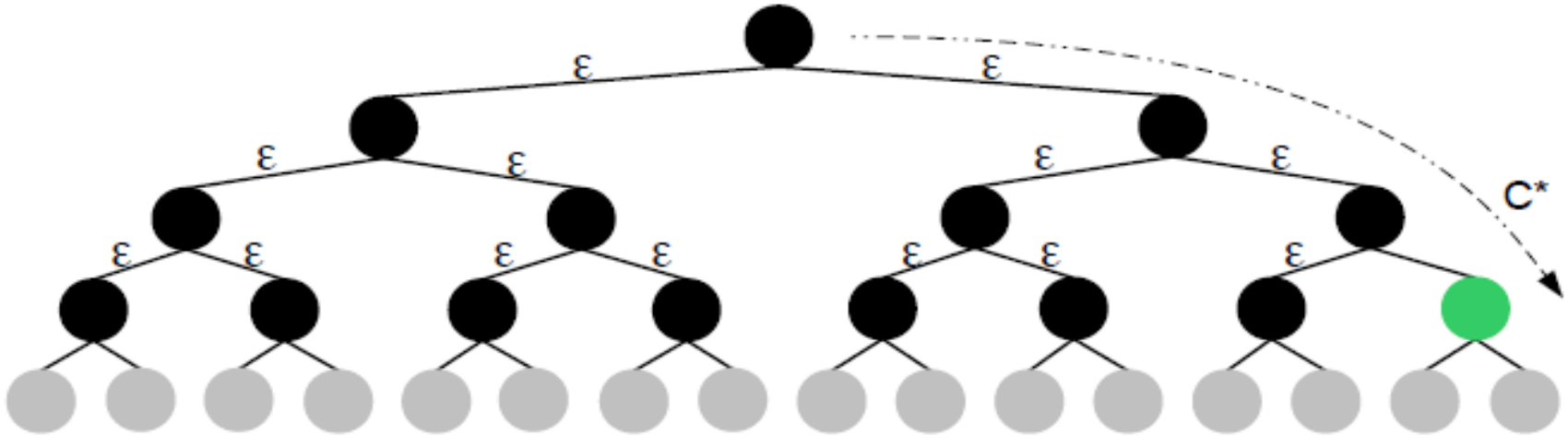
Uniform-cost search properties:

- Complete (if b is finite).
- Optimal.
- Time complexity = $O(b^{1+\lfloor \frac{C^*}{\varepsilon} \rfloor}) \geq O(b^d)$.
- Space complexity = $O(b^{1+\lfloor \frac{C^*}{\varepsilon} \rfloor}) \geq O(b^d)$.

Uniform-cost search

Uniform-cost search properties:

- Upper-bound case: goal has path cost C^*, all other actions have minimum cost of ε:
  - Goal is detected once goal node is popped from frontier.
  - Depth explored before popping goal node: $1 + \lfloor \frac{C^*}{\varepsilon} \rfloor$.
  - Number of generated nodes: $O(b^{1+\lfloor \frac{C^*}{\varepsilon} \rfloor})$.
  - Space and time complexity: all generated nodes.



Uninformed Search

Uniform-cost search:

- A simple extension of BFS that works for any step-cost function.

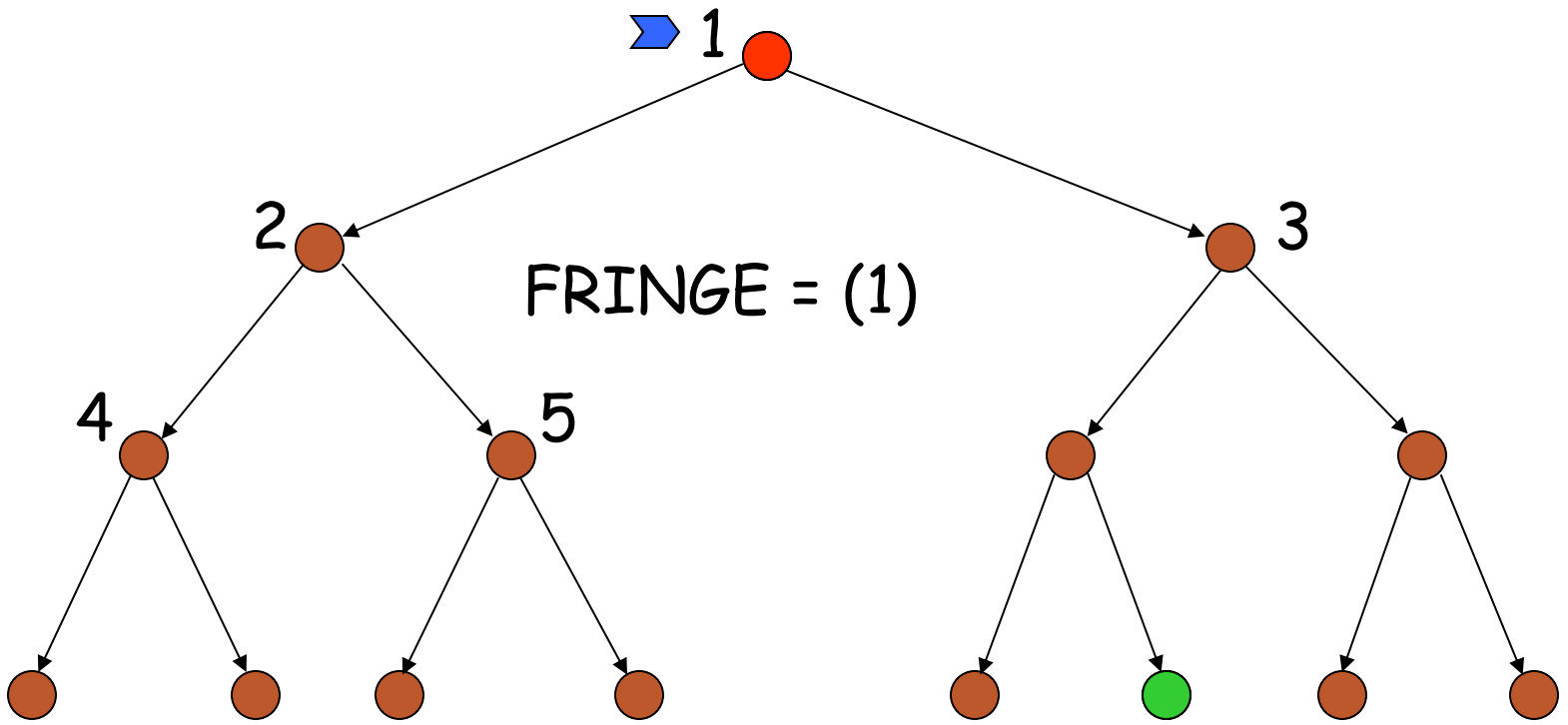
- Instead of expanding the shallowest node, UCS expands the node n with the lowest path cost g(n).
- The frontier is implemented as a priority queue ordered by g.
- Condition: No zero-cost or negative-cost edges.
- UCS algorithm differs from BFS algorithm in 2 aspects:
  - Goal test is applied to a node when it is selected for expansion.
  - In graph version, a test is added in case a better path is found to a node currently on the frontier.

Depth-first search

- Algorithm is similar to BFS, except that the frontier is implemented as a LIFO queue (i.e., stack).
- Algorithm always expands deepest unexpanded node in current frontier.
- Once a leaf is reached, the search backs up (i.e., backtracks) to the next deepest node that still has unexplored successors.
- Search continues until reaching the goal or no frontier nodes are left for expansion.

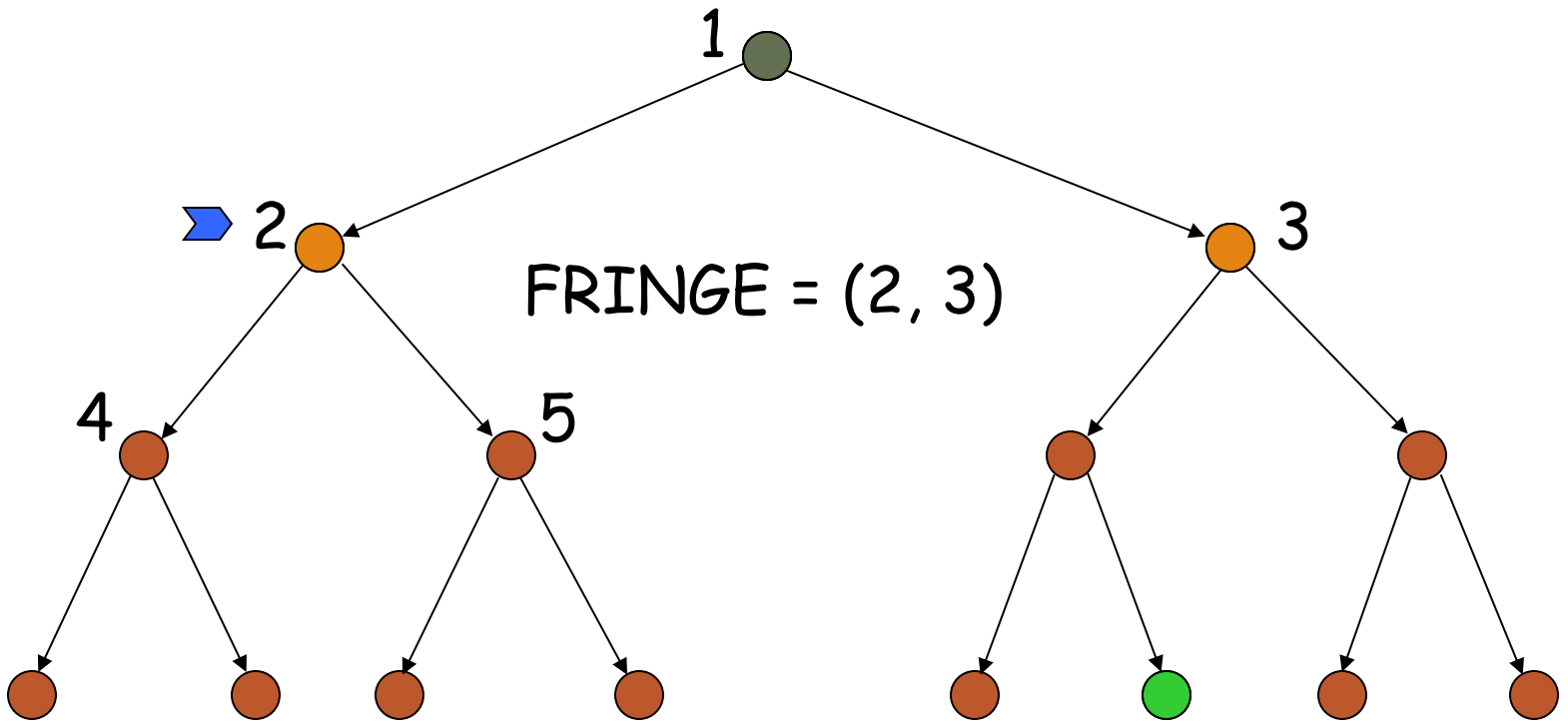
Depth-First Strategy

New nodes are inserted at the front of FRINGE



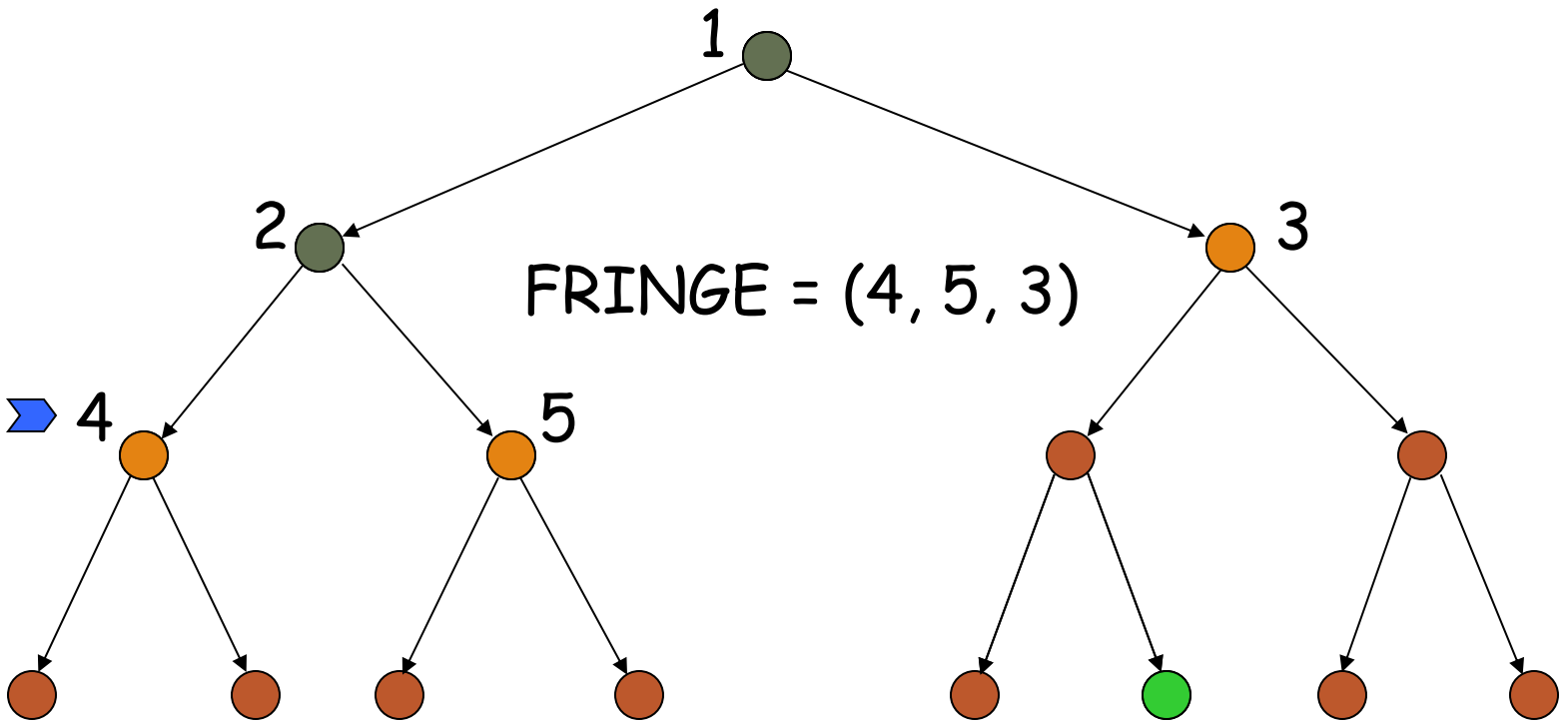
Depth-First Strategy

New nodes are inserted at the front of FRINGE



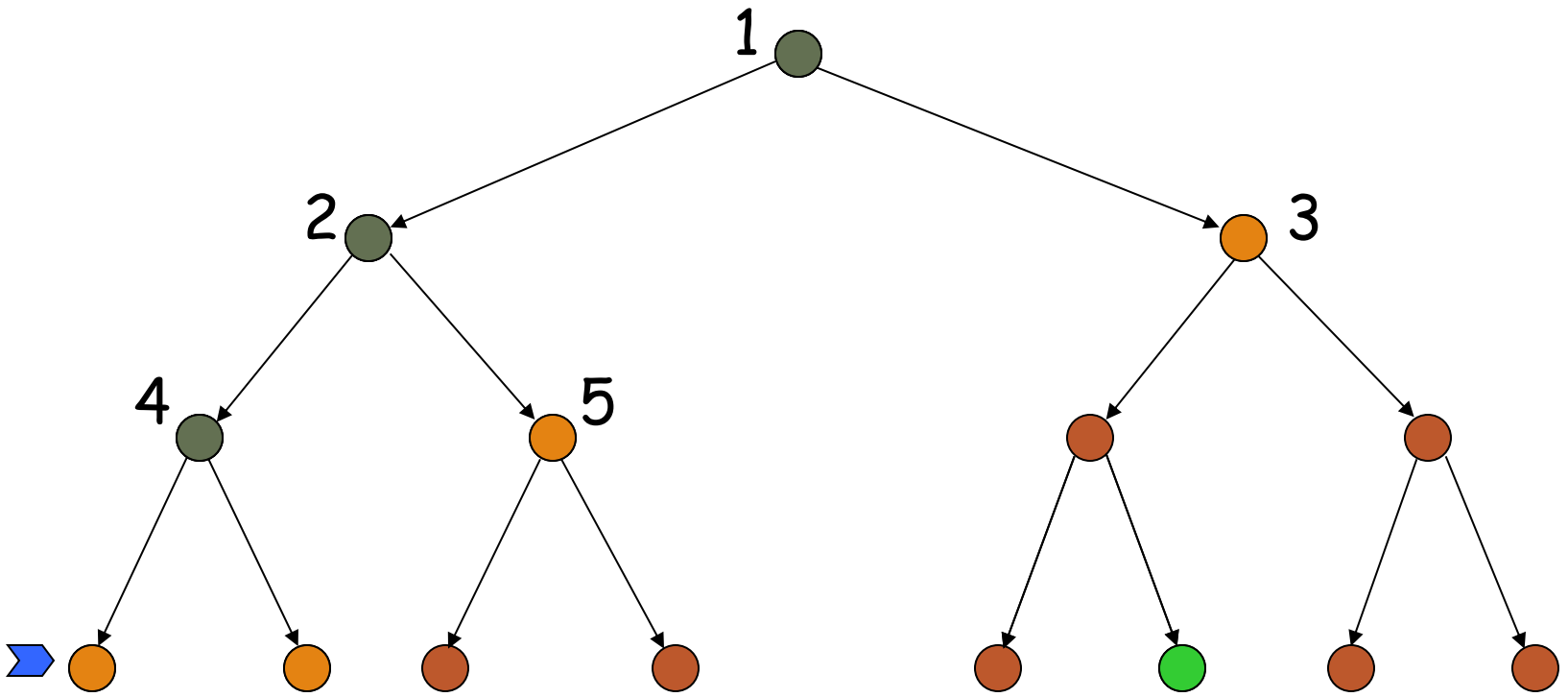
Depth-First Strategy

New nodes are inserted at the front of FRINGE

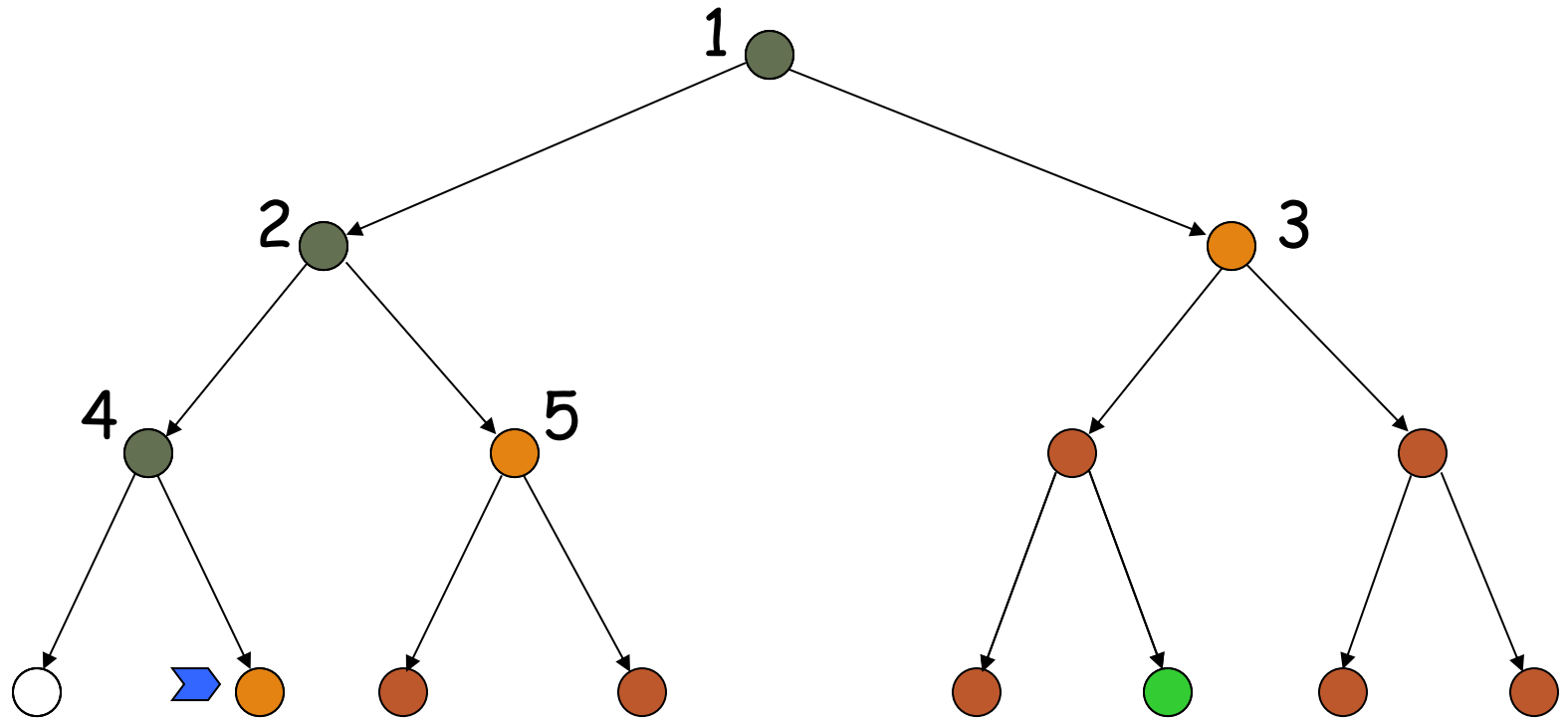


Depth-First Strategy

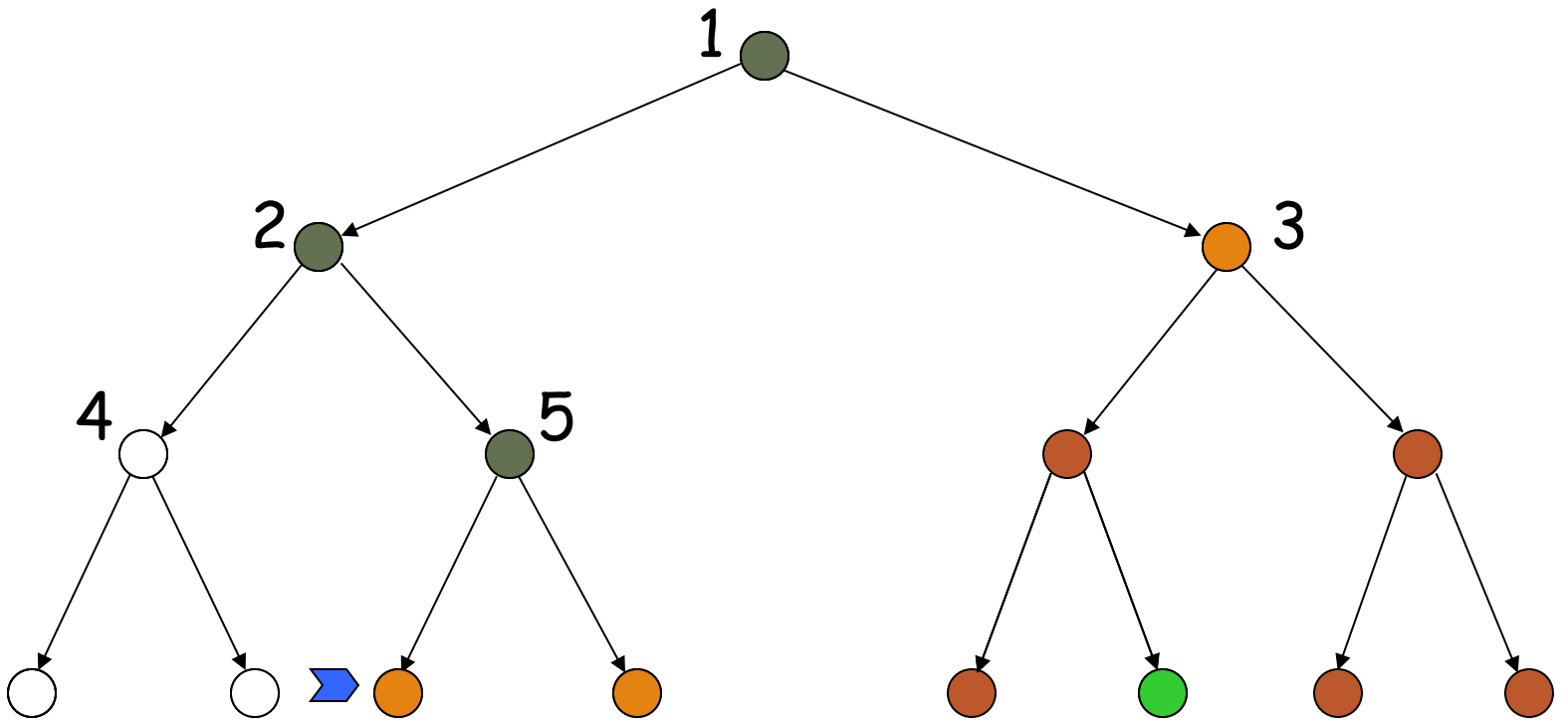
New nodes are inserted at the front of FRINGE



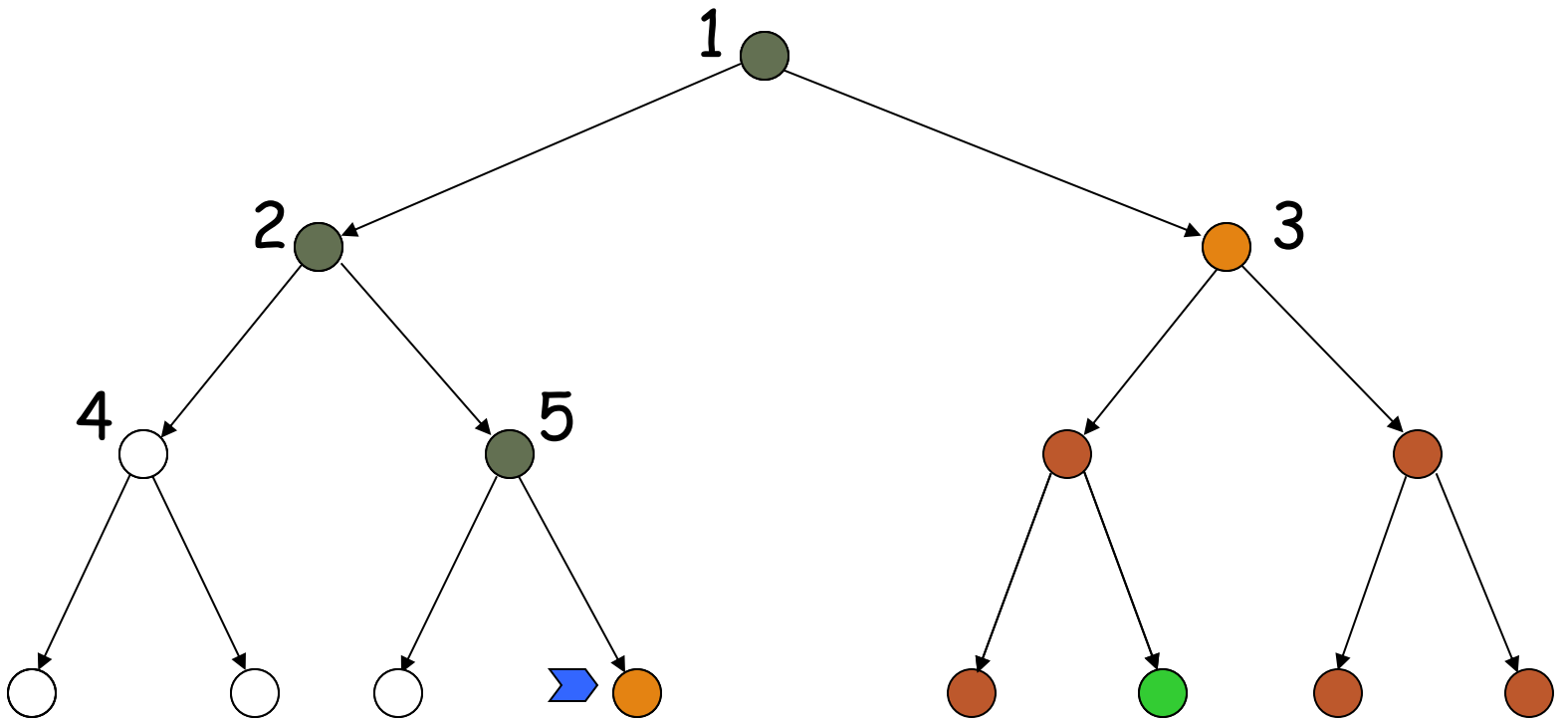
Depth-First Strategy



Depth-First Strategy

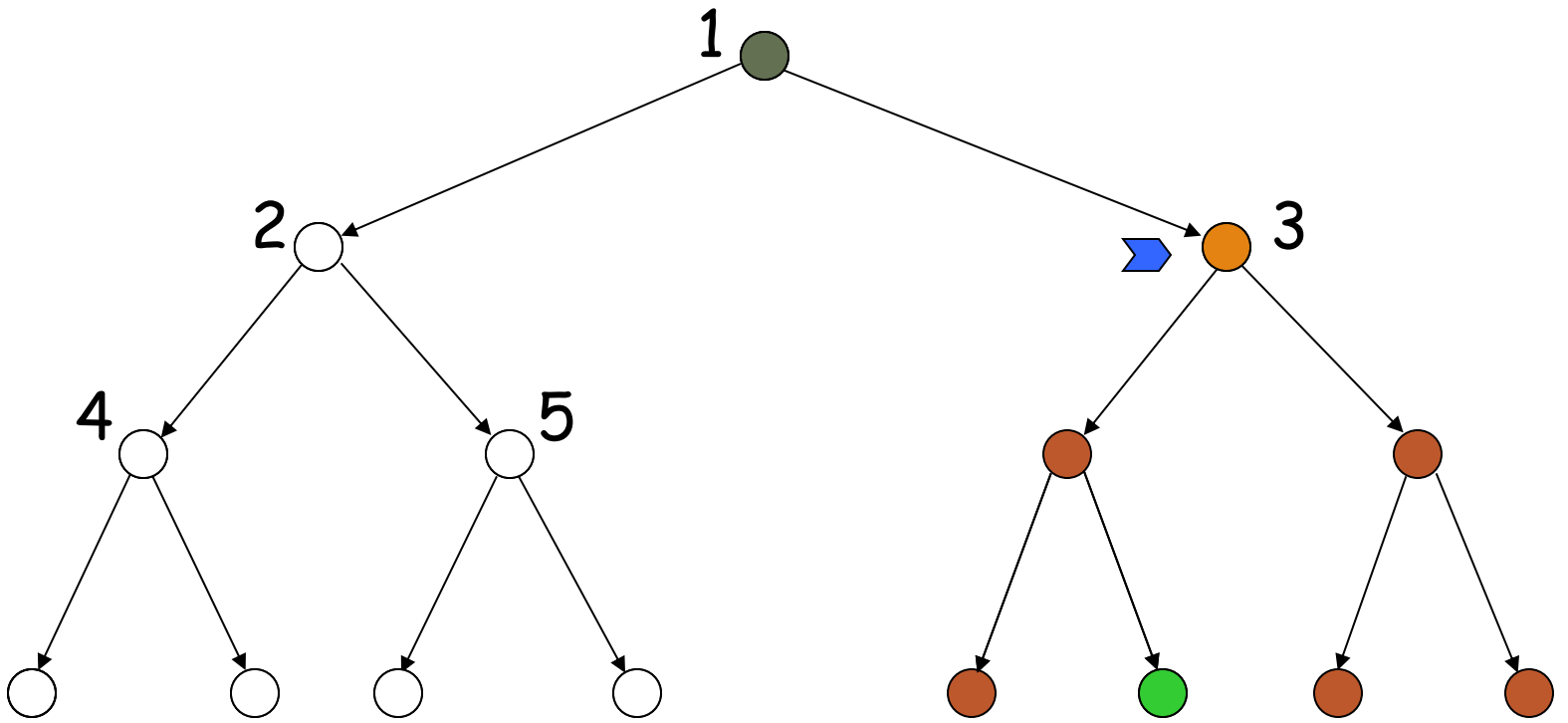


Depth-First Strategy



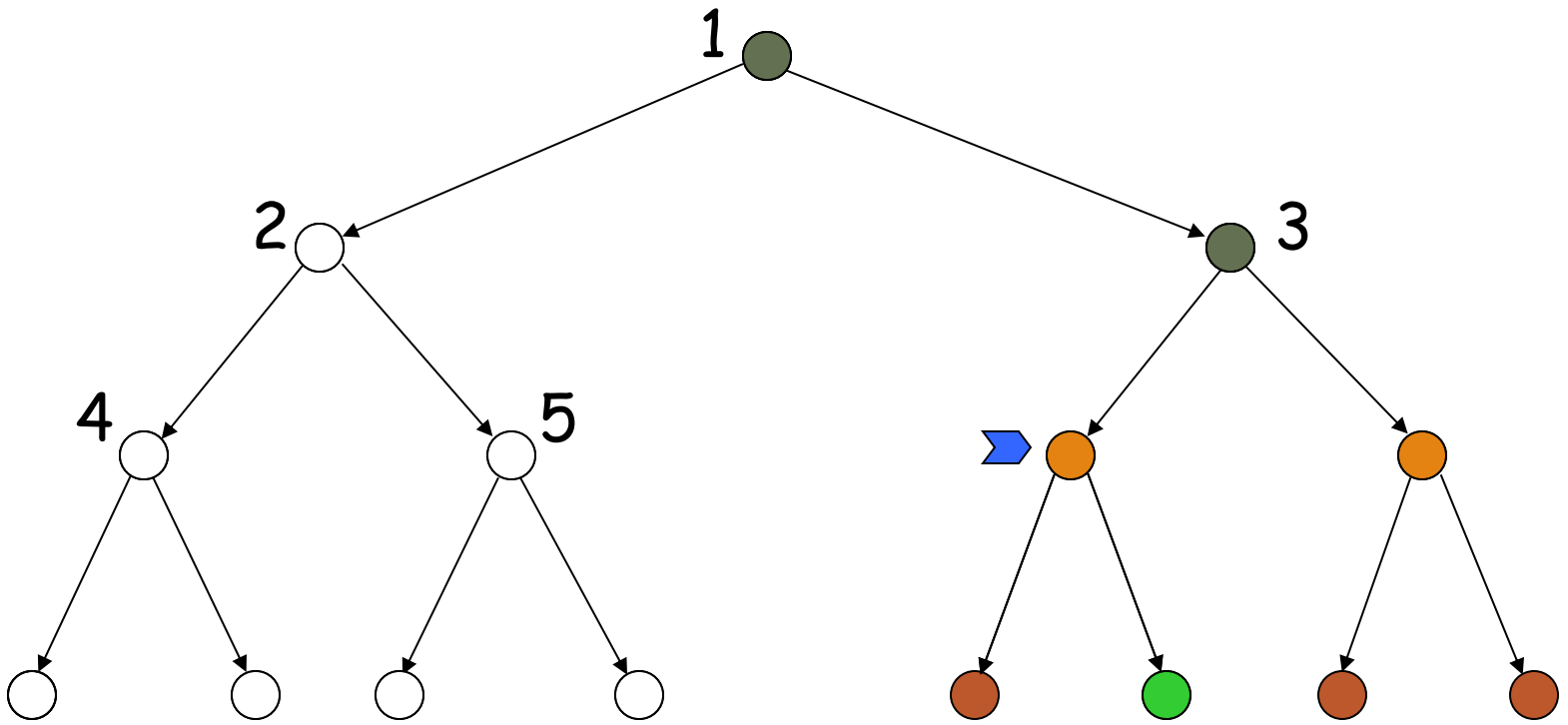
Depth-First Strategy

New nodes are inserted at the front of FRINGE



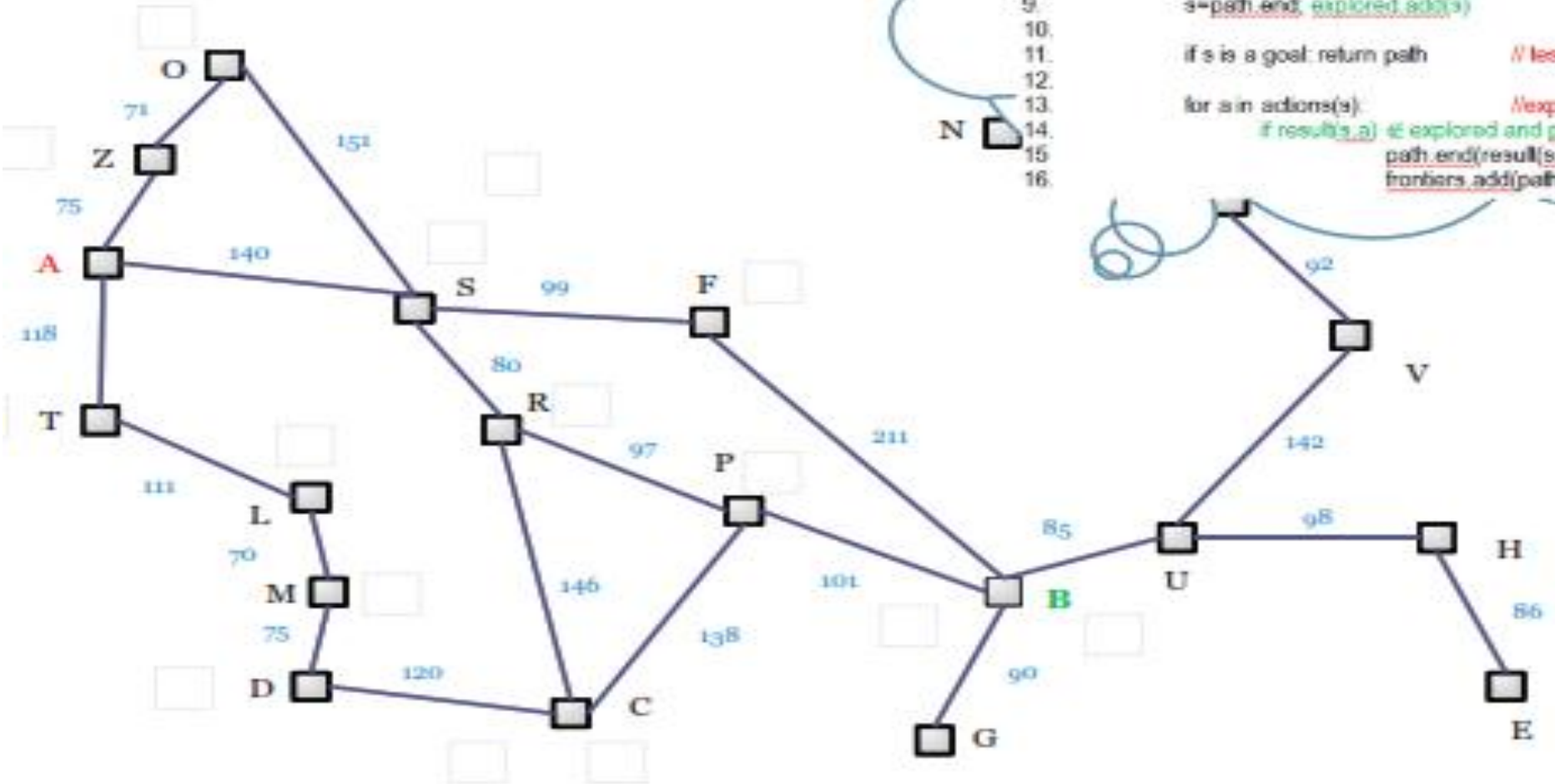
Depth-First Strategy

New nodes are inserted at the front of FRINGE



longest path first search

```
1. GraphSearch(problem)
2.
3.      frontiers={initial}; explored={}
4.
5.      loop:
6.          if frontier is empty: return Fail
7.
8.          path = frontiers.remove()          #choose frontier
9.          s=path.end; explored.add(s)
10.
11.         if s is a goal: return path        # test goal
12.
13.         for a in actions(s):               #expand the path
14.             if result(s,a) ∉ explored and path.end(result(s,a)) ∉ frontiers
15.                 path.end(result(s,a))
16.                 frontiers.add(path)
```



DFS properties

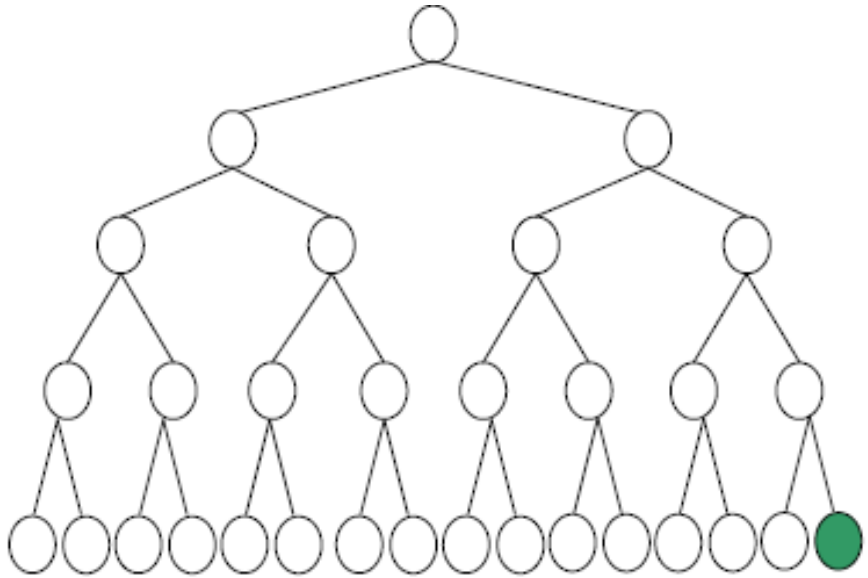
- b: branching factor

- d: depth of shallowest goal node
- m: maximal depth of a leaf node
- Depth-first search is:
  - Complete only for finite search tree
  - Not optimal
- Number of nodes generated (worst case):
  $1 + b + b^2 + \ldots + b^m = O(b^m)$
- Time complexity is $O(b^m)$
- Space complexity is $O(bm)$ [or $O(m)$]

[Reminder: Breadth-first requires $O(b^d)$ time and space]

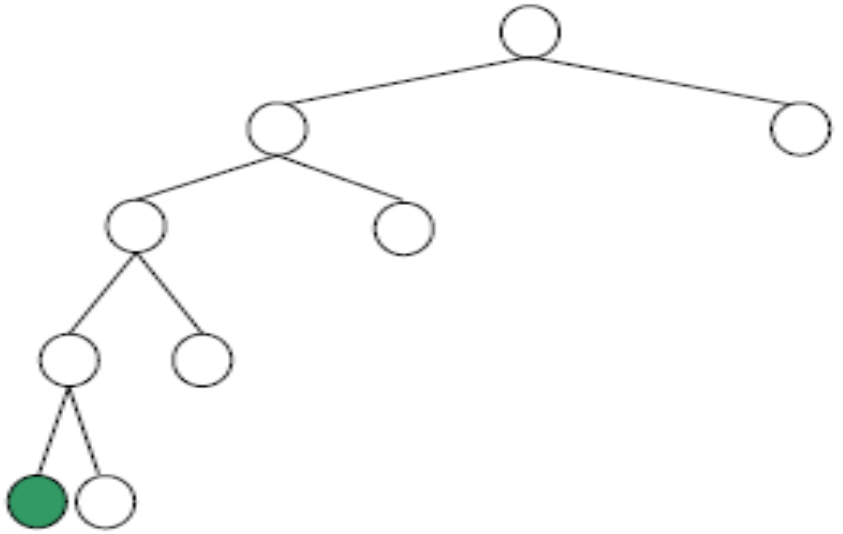
DFS properties:

- Upper-bound case for time: goal is last node of last branch:
  - Number of nodes generated: b nodes for each node of m levels (entire tree).
  - Time complexity: all generated nodes $O(b^m)$.



DFS properties

- Upper-bound case for space: goal is last node of first branch:
  - Number of generated nodes: b nodes at each of m levels.
  - Space complexity: all generated nodes = O(bm).



Depth-limited search

- Depth-first search with depth limit L.
- Algorithm is a simple modification to the general tree-search or graph-search algorithm
  - It takes L as an extra argument.
  - It returns one of the following:
    - solution (solution found with the depth limit).
    - failure (no solution found in entire tree; $l = m$).
    - cutoff (no solution found within depth limit; $l < m$).
- Avoids problems of depth-first search when trees are unbounded.
- Depth-first search is depth-limited search with $L = \infty$

Depth-limited search

- Not complete (unless L = d).
- Not optimal (unless L= d).
- Time complexity = $O(b^L)$.
- Space complexity = O(bL).

Iterative deepening search

- Depth-first search with increasing depth limit L: repeat
- depth-limited search over and over, with L= L + 1.
- Avoids problems of depth-first search when trees are unbounded.
- Avoids problem of depth-limited search when goal depth d > l.

```
function ITERATIVE-DEEPENING-SEARCH(problem)

 for depth=0 to ∞ do

   result   DEPTH-LIMITED-SEARCH(problem, depth)

   if result ≠ cutoff then return result
```

Iterative deepening search

- Complete (if b is finite).
- Optimal.
- Time complexity = $O(b^d)$.
- Space complexity = O(bd).
- Note: nodes on levels above d are generated multiple times.

Summary of Uniformed search

Summary of uninformed search algorithms

Criterion	BFS	UCS	DFS	DLS	IDS
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]
Optimal?	Yes[3]	Yes	No	No	Yes[3]
Time	$O(b^d)$	$O(b^{1+\lfloor \frac{C^*}{\varepsilon} \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor \frac{C^*}{\varepsilon} \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$

- Note 1: assuming finite branching factor b.
- Note 2: assuming minimum step cost $\varepsilon > 0$.
- Note 3: assuming equal step costs.

Requirements

What do I need from you

- When given a certain problem you should be able to:
  - Formulate the problem:
  - What is a state?
  - What is the initial state?
  - What actions could be applied to a state?
  - What is the transition model?
  - What is the goal test?
  - What is the step cost as well as the path cost?
- Build the search tree up to a given depth.
- Traverse the search tree according to a given strategy.
- Answer descriptive questions.

QUIZ

- List the abbreviation for the following:-
  - BFS
  - UCS
  - DFS
  - DLS
  - IDS

QUIZ

- List the different types of agents then explain any one of them?
- Explain in details the STATE-SPACE problem formulation

QUIZ

- List the different types of agents then explain any one of them?

  - Simple reflex (aka reactive, rule-based)
  - Model-based
  - Goal-based
  - Utility-based (aka decision-theoretic, game-theoretic)
  - Learning (aka adaptive)

QUIZ

- Explain in details the STATE-SPACE problem formulation
  - A problem can be defined formally by five components:
  - 1) The initial state that the agent starts in.
  - INITIAL-STATE : STATE
  - 2) The possible actions available at each state.
  - ACTIONS : STATE → ACTION SET
  - 3) The transition model describing what each action does.
  - RESULT : STATE x ACTION → STATE
  - 4) The goal test that detects the goal state.
  - GOAL-TEST : STATE → BOOLEAN
  - 5) The path cost expressed in terms of step cost.
  - STEP-COST : STATE x ACTION → REAL+.